ANALYSIS REPORT

Harmonic Regression Model Explanatory Report

Feedforward Transmission Error Baseline

Styled PDF edition generated from the canonical Markdown analysis report for improved readability, section hierarchy, and table presentation.

Overview

This report documents the implemented `harmonic_regression` model for TE prediction.

This model is structurally different from the plain feedforward baseline because it encodes the periodic nature of TE directly in the functional form instead of asking a generic neural network to discover it implicitly.

Model Description

`harmonic_regression` is a structured regression model based on a truncated harmonic expansion of angular position.

Its core idea is

- TE is modeled as a sum of harmonic basis functions of the angular position;
- the harmonic coefficients are either globally shared or conditioned by operating variables.

The current implementation supports two modes:

- `static`

  One global coefficient vector is learned for the whole dataset.
- `linear_conditioned`

  The coefficient vector is adjusted linearly from the operating-condition inputs. This makes the model much more interpretable than a generic MLP.

Operating Principle

The model assumes that the main TE structure can be approximated by:

```
TE(theta) = a0 + sum_k [a_k sin(k theta) + b_k cos(k theta)]
```

In practice

1. angular position is converted from degrees to radians;
2. harmonic features are generated up to the configured order;
3. a coefficient vector is resolved;
4. the final prediction is the inner product between features and coefficients.

In `static` mode:

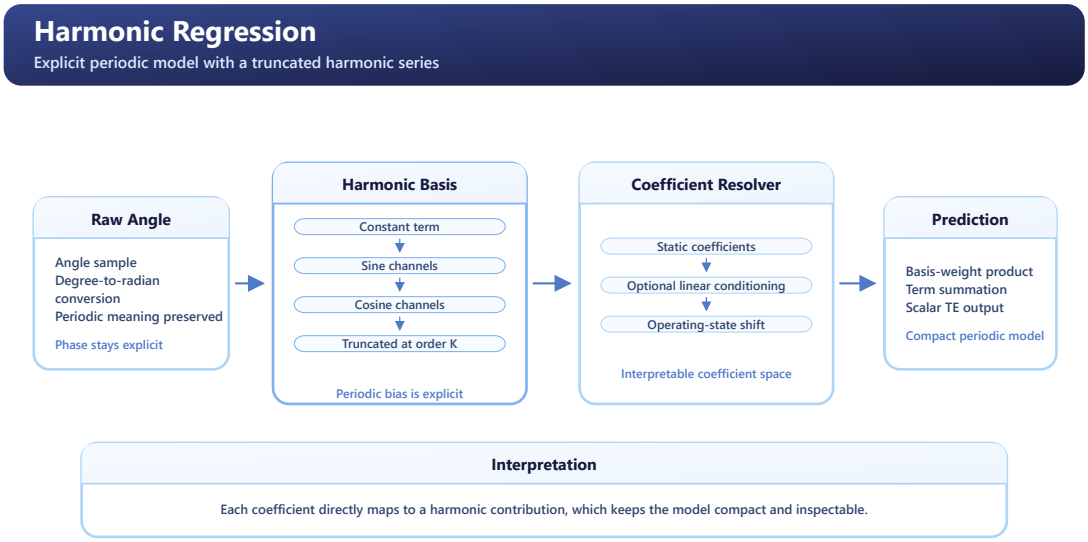
- one shared coefficient vector is learned.

In `linear_conditioned` mode:

- the shared base coefficients are corrected by a linear projection from the non-angular input features.

This is the first explicitly TE-structured model in the repository.

Conceptual Map



Harmonic regression conceptual diagram

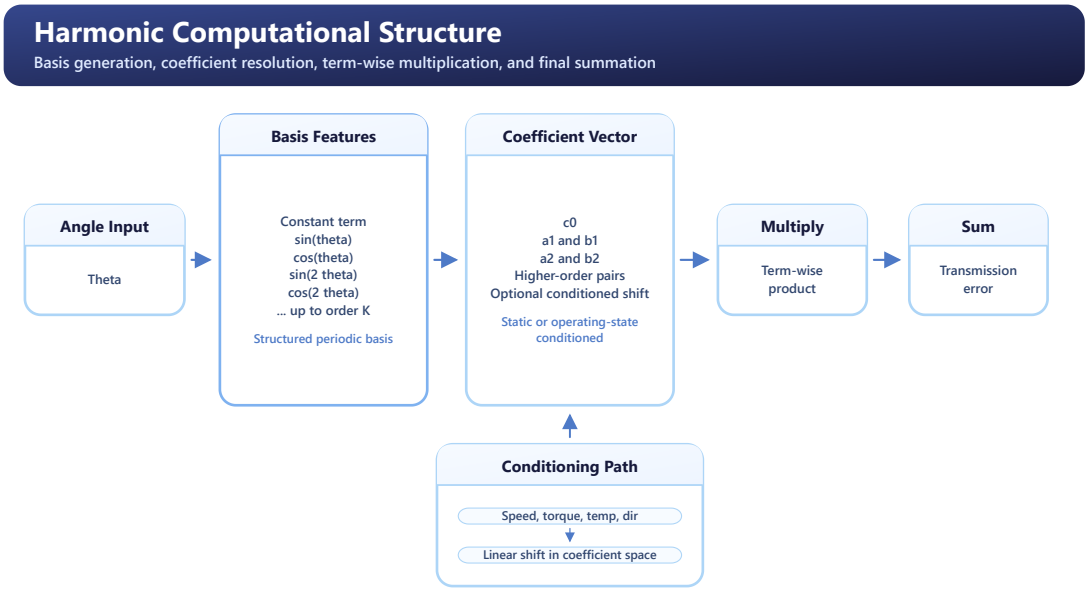
The model can be summarized as

```text
Input Point -> raw angle theta -> [1, sin(theta), cos(theta), ..., sin(K theta), cos(K theta)] -> harmonic coefficient vector -> element-wise product -> sum -> scalar TE prediction
```

Conditioned variant

```text
[speed, torque, temperature, direction] -> linear projection -> coefficient correction -> base coefficients + correction -> harmonic prediction
```

Architecture Diagram



Harmonic regression architecture diagram

This architecture-style diagram should be read as a computational graph rather than as a dense neural network.

The main blocks are

- angle input;
- harmonic basis generation;
- coefficient vector resolution;
- term-wise product;
- final summation into the TE output.

Why This Model Exists

This model exists because TE is fundamentally periodic with respect to angular position.

Instead of learning periodicity indirectly, the model states it up front and only learns:

- how strong each harmonic is;
- whether operating conditions should shift those coefficients.

This gives a compact and interpretable baseline for the structured-static wave.

Advantages

- Strong inductive bias aligned with periodic TE behavior.
- Highly interpretable coefficients.
- Very compact parameterization.
- Easier to inspect and reason about than a generic neural network.
- Strong candidate for PLC-friendly deployment if performance is adequate.

Disadvantages

- Limited expressiveness outside harmonic structure.
- Assumes the main TE shape is well captured by a truncated series.
- Conditioning is currently only linear in the conditioned variant.
- May underfit nonlinear operating-condition effects that are not well expressed through coefficient shifts.

Expected Behavior In The TE Context

This model is expected to work well when:

- TE is dominated by periodic components;
- the harmonic order is sufficient;
- operating-condition dependence is moderate or smoothly varying.

It may struggle when:

- residual nonlinear behavior is strong;
- condition dependence is highly nonlinear;
- the TE signal contains effects that are not well represented by the chosen harmonic truncation.

Python Model Implementation

The implementation lives in `scripts/models/harmonic_regression.py`.

`HarmonicRegression.__init__(...)`

This constructor defines the structure of the regression model.

Main responsibilities

- validate that the input contains the TE operating-condition channels;
- validate scalar output and positive harmonic order;
- store the coefficient mode;
- compute the harmonic feature count;
- create:
  - `base_coefficient_tensor`
  - optional `conditioning_projection`

    The key design choice is that the structured part is explicit and extremely small compared with a neural network.

`build_harmonic_feature_tensor(...)`

This function constructs the actual harmonic basis.

It

- converts angle from degrees to radians;
- starts with the constant term;
- appends `sin(k theta)` and `cos(k theta)` for each harmonic order.

  This is the mathematical core of the model.

`resolve_coefficient_tensor(...)`

This function decides how the harmonic coefficients are obtained.

In `static` mode

- it replicates the shared coefficient vector across the batch.

  In `linear_conditioned` mode:
- it adds a linear condition-dependent correction to the shared base coefficients.

  This is what gives the model its two variants.

`forward_with_input_context(...)`

This is the repository-specific forward interface used by structured models that need both:

- raw input features;
- normalized condition features.

  The function:
- takes raw angle from `input_tensor`;
- takes normalized operating conditions from `normalized_input_tensor`;
- builds the harmonic basis;
- resolves the coefficients;
- computes the final weighted sum.

  This is why the model does not use the plain `forward(normalized_input_tensor)` interface of the generic MLP.

Repository Integration

The model is registered in `scripts/models/model_factory.py` under `model_type == "harmonic_regression"`.

This registration makes it selectable from YAML and compatible with the shared training path.

Training Workflow Overview

Although the model is not an MLP, it still uses the same structured-neural training entry point in `scripts/training/train_feedforward_network.py`.

At high level

1. the dataset is still flattened into point-wise samples;
2. normalization statistics are still computed;
3. the model is instantiated by the factory;
4. the Lightning regression module detects that the backbone exposes `forward_with_input_context(...)`;
5. training proceeds with the same optimizer, logging, checkpointing, and registry machinery.

Training Logic Relevant To This Model

`TransmissionErrorRegressionModule.forward_regression_model(...)`

This is the key adaptation point for `harmonic_regression`.

The function checks whether the backbone implements:

- `compute_auxiliary_output_dictionary(...)`, or
- `forward_with_input_context(...)`

  Since `HarmonicRegression` provides `forward_with_input_context(...)`, the regression module passes both:

- the raw input tensor;
- the normalized input tensor.

  This is necessary because:

- the angular harmonic basis must be generated from the raw angle in degrees;
- the operating-condition channels are better handled in normalized form.

`train_feedforward_network(...)`

This function is still the main trainer for this model family.

Nothing in the outer training loop is harmonic-specific. The specialization happens inside the backbone and the regression module interface.

This is a strong architectural choice in the repository:

- specialized model behavior;
- shared training infrastructure.

Practical Interpretation

`harmonic_regression` should be read as the first serious structured baseline:

- much more interpretable than the plain feedforward network;
- much less flexible than a general neural model;
- probably strong when periodicity is dominant;
- a useful reference for measuring whether residual neural corrections are actually needed.